A PRACTITIONER'S GUIDE

Agentic AI for Empirical Research

A Practical Workflow, and How to Verify What It Produces

Claes Bäckman

Leibniz Institute for Financial Research SAFE
claes.backman@gmail.com • claesbackman.github.io

AI agents can now read a research project, write and run code, edit drafts, and iterate until a task is finished. They are fast, cheap, and competent in languages the researcher does not know. This shifts the binding constraint in empirical work from producing analysis to establishing that the analysis is right.

This guide covers both halves of that problem. The first half is a practical workflow: where these tools run, how to set up a project so an agent is useful instead of merely busy, which file formats need converting, how to record project context so it does not have to be re-explained every session, how to choose a model without paying for capability the task cannot use, and how to work when the data is held under a confidentiality agreement.

The second half, which is the part I think matters more, is verification. A cost-ordered set of checks for establishing that AI-generated code and analysis are correct before they enter a paper, and an argument that the cheapest verification happens while the code is being written.

Nothing here is a benchmark or a model evaluation. It is a working account of how to use these tools on an empirical paper without losing the ability to defend what the paper says.

Version 1.0 • August 2026. This document is assembled from a set of guides maintained at claesbackman.github.io/ai-guides.html, where the tool-specific material is kept current. Section 2, parts of Section 4, and Appendix B describe software and pricing that change every few months, and should be read against the dated web version. The argument in Sections 5–7 is meant to outlast the products it was written about.
Corrections, disagreements, and additional checks are welcome. This is intended as a living document.

Contents

1 Introduction

Verification is becoming a bottleneck for empirical research. AI agents are fast, cheap, and competent at producing code, including in languages that you may not know (I do not really know how to build a website, but Claude does). While I have seen plenty of concern about verification, I have not seen a practical guide that collects ways of actually verifying what these tools hand back.

This document is an attempt at one, wrapped in enough workflow material to be useful to someone who has not yet started. It is written for academic economists, and the examples are drawn from that work — panel construction, regression tables, LaTeX drafts, administrative data — but very little of it is specific to economics.

1.1 Who this is for

Roughly, four groups.

- **People who have not started.** Sections 2 and 3 are for you. Where these tools run, and what to do in the first hour so that the tool becomes useful on a research project instead of a toy.
- **People already using an agent daily.** Skip to Section 5. The workflow material will be familiar. The verification ladder in Section 6 probably contains at least one check you are not running.
- **People working with administrative or otherwise restricted data.** Read Section 8 first. It changes what the rest of the document means for you, and the setup it describes is much easier to do before the first session than after.
- **People who referee, supervise, or co-author with the above.** Section 6 is a reasonable starting point for what it is fair to expect someone to have done before showing you a table.

1.2 How to read it, given that it will go out of date

A guide to specific software has a short shelf life. Product names, approval modes, and file conventions in this area change on a timescale of months. I have tried to handle that by separating two kinds of content.

The body of the document is about the parts I expect to remain true: that agents produce plausible-looking output, that the errors which survive their own debugging are the invisible ones, that independence between checks has to be enforced structurally rather than requested, and that being present while code is written is cheaper than reconstructing it afterwards. None of that depends on which vendor you use.

Appendix B holds the material that decays. The side-by-side comparison of Claude Code and Codex, the specific file names, the approval-mode vocabulary. It is stamped with the date the official documentation was checked. When it conflicts with the web version of these guides, the web version is right. The same caution applies to the discussion of model tiers and pricing in Section 4, where the reasoning should hold up better than any specific number in it.

> **Two cautions before anything else.** First, data subject to a confidentiality agreement does not go into an agent. Note that installing the tool on your own laptop is not sufficient, because the model still runs on the vendor's servers and reads whatever you point it at. Section 8 is about what to do instead, and it is worth reading before your first session if you work with administrative registers. Second, the model's confidence is not calibrated. Verify regression results, citations, and identification claims yourself before they enter a

> draft. The rest of this document is largely an elaboration of that second sentence.

2 Where these tools run

Claude Code and Codex are the two main agentic systems for everyday research work. Both can read your files, run code, edit drafts, and iterate on a task until it is done. The choice between them is mostly a matter of taste, existing accounts, and what your collaborators or employer already use. Appendix B works through the differences.

The more consequential early decision is not which system but which *surface*, meaning where the agent runs. There are six, and most researchers end up using two or three.

2.1 VS Code

VS Code is an editor where you can write and run code, work in LaTeX, and keep your project files in view. Both systems ship official VS Code extensions, where the agent sits in a side panel, sees the file you have open, can edit anywhere in the project, run code, view tables and figures, and show you diffs before you accept them. If you want to see your files in the same window as the agent, this is where to start.

There is a learning curve to VS Code itself. Section 3 covers the first day specifically for empirical research.

2.2 Terminal

Both systems also run as command-line tools in any shell. You point the tool at a folder, describe what you want, and it edits files in place. Under the hood this is the same thing as the VS Code version without the editor wrapper. It is the natural fit for anyone already working in a terminal.

Two cases where the terminal clearly beats the editor:

- **Remote servers.** SSH into a cluster and run the agent there. Necessary when your data lives behind a firewall — a national statistics agency, a secure research environment — and cannot be moved to your laptop.
- **Background runs.** Long jobs you want to walk away from. An agent can run unattended for hours, building a robustness table over twelve specifications or converting a folder of PDFs to markdown, and leave you a diff to review.

2.3 Desktop applications

The vendor desktop apps are the chat-style interfaces most people meet first, but both now include modes built for sustained work as well as conversation: background task runners, and dedicated interfaces for starting an agent session on a folder or repository and reviewing the resulting diffs.

For quick conversation and drafting, ordinary chat remains easiest. For sustained editing, use a code-oriented surface or an IDE, because chat-only surfaces become hard to track once many files and diffs are involved.

2.4 Web

For ordinary chat the browser versions are close to the desktop apps. The exception worth knowing about is that both vendors run long-lived cloud agents from the web. You point one at a GitHub repository, describe the task, and it works in a cloud container for minutes or hours before pushing a branch or opening a pull request for you to review.

Cloud agents are useful in three situations: when your work laptop blocks software installation, which is common at central banks and statistical agencies, when you want to start a job from a borrowed machine, and when you want several independent attempts at the same task running in parallel. They are useless when the data lives only on your laptop or inside a secure environment, because the cloud agent cannot see it.

2.5 Mobile

You will not edit a Stata script on a phone. Two patterns do work. *Capture*: dictate an idea, a literature note, or a draft paragraph while walking. Voice input is now good enough that talking through a paper's argument and getting structured notes back is genuinely practical. *Monitor*: start a cloud agent from your laptop in the morning, then approve or redirect it from your phone later. You can review a pull request, leave a comment, and ask the agent to revise, without opening a laptop.

2.6 Other editors

If you already work in another editor you do not have to switch. Cursor is a VS Code fork built around AI from the ground up. The JetBrains IDEs have first-party integrations for both agents, and in a fast editor like Zed you can run either command-line tool from the integrated terminal. If you have no strong preference, VS Code remains the path of least resistance, because it is what most documentation, screenshots, and shared workflows assume.

2.7 Which surface for which task

Surface	Best for	When not to use
VS Code	Daily coding, paper writing, anything where you want a real editor and a diff view.	Quick conversational work. Overkill for "explain this paragraph to me."
Terminal	Remote servers, secure data environments, long unattended jobs, scripting an agent into a pipeline.	If you are not already a terminal user, the learning curve is not worth it just for this.
Desktop	Focused workspace, visual diff review, parallel sessions, quick drafting.	When you need a full editor around the agent, or the work must run on a remote or secure server.
Web	Borrowed machines, locked-down laptops, starting cloud agents on a repository.	Data that cannot leave your machine or a secure environment.
Mobile	Capturing ideas on the move, reviewing or redirecting cloud agents, voice notes.	Any actual editing. Phones are for triage, not for work.
Other IDEs	Sticking with the editor you already know.	If your editor only offers terminal access and you want rich diff review.

A reasonable starting point. Install an agent inside VS Code for daily editing, keep a desktop app open for drafting and exploration, and learn the command-line version once you have a job that needs to run on a server or in the background. Web and mobile are useful additions but not where most of the work happens.

3 Setting up a research project

The tools work out of the box. A few one-time steps make them substantially more useful on an empirical project, and skipping them is the most common reason people try an agent once and conclude it is not for them.

3.1 Open the folder, not the file

Open your whole project directory, not a single script. The agent needs the directory tree to be useful, so that it can see that `01_clean.R` produces the file that `02_analysis.R` reads, that tables land in `paper/tables/`, that the raw data is where you say it is. This can be a git repository, a synced folder, or a plain folder on your desktop. All work equally well.

3.2 Install the extensions for your stack

If you work in VS Code, install the extensions for the languages you use before you start. The agent reads what is installed, and can then run code, render tables, and preview PDFs directly. The specific extension identifiers are listed in Appendix B. The reasoning behind the choices is here.

For **Stata**, the relevant extension lets Stata code run from the editor and exposes an interface the agent can drive directly, so it can execute commands, inspect variables, and read stored results, so it is not writing do-files blind.

For **Python**, install the core extension, a language server, the debugger, and environment management. This is worth doing *even if you do not write Python*, which is the part people skip. A surprising share of the everyday friction in empirical work — converting a PDF or a Word file to markdown, reshaping a messy CSV, scraping a table from a webpage, pulling a series from an API, cleaning a bibliography file, splitting a 200-page document into chapters — is solved fastest by a ten-line Python script. You do not need to know the language to benefit. Ask in plain English ("convert this PDF to markdown, keep the tables", or "download monthly CPI from FRED and save as CSV") and the agent will write, run, and debug the script. Installing the Python extensions is what makes that path smooth: the agent can execute the script, see the error, and iterate without you setting anything up. Think of Python less as a language you need to learn and more as general-purpose glue the agent uses on your behalf.

For **LaTeX**, a compile-on-save extension with a side-by-side preview pairs well with an agent for table and notation fixes, because you see the output update immediately. For **data files**, a CSV colouriser and an inline PDF viewer let you sanity-check raw data and read codebooks without leaving the editor.

3.3 What the agent can read, and what to convert first

These tools work best with plain text. The more a file looks like characters-on-a-page and less like a rendered binary, the better the agent can read, edit, and reason about it. For economists this has practical consequences: a referee report in `.docx` or a codebook as a scanned PDF are both usable, but a text version of the same file produces noticeably better results.

Format	Handling
`.md .txt .tex .bib .csv .json`	Native, no friction. Read and edited line by line. Markdown and LaTeX are the ideal substrate for any writing task. CSV is fine up to a few thousand rows. Larger files should be summarised or sampled first.
`.py .R .do .jl`	Native. Read, edited, and run. Stata do-files are read as text even without a Stata installation, though executing them requires the Stata extension.
`.ipynb`	Native. Notebook cells — code, markdown, and stored output — are read and edited directly, without round-tripping through a script.
`.docx`	Usable with a conversion step. Formatting, comments, and tracked changes come through awkwardly. For referee reports or co-author comments, convert to markdown first with `pandoc file.docx -o file.md`. The agent will do this for you.
`.xlsx .xls`	Convert to CSV first, because the binaries cannot be read directly. Split multi-sheet workbooks into one clearly named CSV per sheet.
`.dta .rds .sav`	Convert first. Ask for a short script — Stata `export delimited`, R `write_csv`, Python `pyreadstat` — to dump the dataset, a sample, or a summary to CSV or markdown.
`.pdf`	The most common problem format. See below.
Images	Partial. Plots and screenshots of tables can be viewed directly, which is useful when you want a figure improved. Text cannot be reliably extracted from scans, so run OCR first.
`.qsf` (Qualtrics)	Convert first. Technically JSON, but deeply nested. Ask for a script that flattens it into a markdown outline of blocks, questions, and answer choices.

3.3.1 PDFs

PDFs are where most researchers get stuck. They look like documents, but they are a layout format rather than a text format. The "text" is often a sequence of glyphs positioned on a page, with no inherent notion of paragraphs, columns, or tables. Results vary enormously with how the PDF was produced.

- **Born-digital PDFs** (from LaTeX or Word) are usually fine for short documents, read directly. Expect some mangling of tables, footnotes, and multi-column layouts.
- **Long PDFs** (50+ pages) should be converted to markdown first. Reading a 200-page PDF directly consumes the context window and slows every subsequent turn. A clean markdown version is a fraction of the size and easier to navigate.
- **Scanned PDFs** cannot be read as text at all, because they are images of pages. Run OCR first (`ocrmypdf` is a good free option), then convert.
- **PDFs with equations and tables** are the hardest case, and any extraction will be

imperfect. If you can get the source `.tex`, use that instead. It will always be cleaner than re-parsing the compiled output.

For conversion, `pandoc` is the universal first try and works best on clean born-digital files. The machine-learning-based converters — `marker`, `MinerU`, `docling` — are noticeably better on papers with tables, figure captions, and equations. All are free, and the agent can install and run them for you.

> **Convert once, commit the result.** If a format requires a conversion step, do it once up front and keep the markdown version alongside the original. Every later session then starts from clean text and you do not burn context re-converting the same document. This is especially worth doing for codebooks, data dictionaries, and reference PDFs you will revisit over months.

3.4 Persistent project context

Both systems read a project instruction file automatically at the start of every session: `CLAUDE.md` for Claude Code, `AGENTS.md` for Codex. This is where you record what is always true about the project — conventions, variable naming, dataset structure, what not to touch — so you do not repeat it in every prompt.

The fastest way to create one is to ask the agent to generate a draft by reading the project folder, then edit the result. A useful shape for an empirical project:

```
# Project: [Paper title]

## Data
- Unit of analysis: firm-year panel, 2010-2022
- Raw data lives in data/raw/ - never edit these files
- Main dataset after cleaning: data/clean/panel.dta (N ~ 47,000)

## Code conventions
- R scripts are numbered (01_clean.R, 02_analysis.R, ...)
- All regressions cluster SEs at the firm level
- Use fixest for all panel regressions

## LaTeX
- Main file: paper/main.tex
- Tables generated by 03_tables.R go to paper/tables/
- Use \widehat{} not \hat{} for estimators
```

These files work at two levels, loaded simultaneously. A **user-level** file in your home directory applies to every project on the machine, and is the right place for preferences that never change: your preferred language, how you like standard errors clustered, notation conventions, a note that you work with Scandinavian register data and prefer variable names in English. A **project-level** file at the repository root applies only in that folder, and holds dataset structure, sample restrictions, which script produces which output, and co-author conventions. Where they conflict, the project file wins.

This is also the right place for instructions about how you want the agent to *write*: tone, voice, the hedging and phrasing to avoid. Paul Goldsmith-Pinkham has a useful post on using these instructions to get assistance that improves rather than flattens your prose.[1]

[1] https://paulgp.com/

> **Keep it short.** A project-context file that grows without limit gets followed less closely. Put facts in it, not chatty prose, and point to longer documents instead of inlining them. See the note on `decisions.md` at the end of Section 5.

3.5 Skills: reusable workflows

Both systems let you save a set of instructions as a named, reusable workflow, a markdown file that you invoke by name to run a procedure you repeat across projects. A standard robustness check, a referee-style review, a table reformatting routine.

The mechanics are a folder containing a `SKILL.md` file, where the folder name becomes the command. A minimal example:

```
---
name: robustness
description: Run standard robustness checks on the main regression.
---

For the main specification in this project:
1. Re-run with alternative clustering (industry-year instead of firm)
2. Re-run dropping the top and bottom 1% of the outcome variable
3. Re-run on the pre-2020 subsample only
4. Produce a summary table comparing coefficients across all specs
```

Skills can be personal, in a home directory, or checked into the repository so co-authors share them. There is a growing library of shared skills written by economists: my own collection of research-feedback skills, Scott Cunningham's *MixtapeTools*, and Chris Blattman's set of research-workflow skills.[2] Copying a skill folder into the right directory makes it available immediately.

One thing to know: an agent may invoke a skill on its own when it judges the description relevant, not only when you ask for it by name. If you would rather control when a skill runs, most systems have a flag to disable model invocation. The one for Claude Code is in Appendix A.

3.6 Git

These tools are git-aware. When the project is a repository, the agent can see commit history, staged changes, and diffs without you pasting anything. That makes it useful for the version-control tasks themselves: writing commit messages, summarising what changed between two commits, resolving a merge conflict by reading both sides.

For getting started, git is optional. A synced folder or a plain directory works fine as a project root, and the installation guides say so.

For verification, it stops being optional, and this is the one setup recommendation in this document I would make unconditionally. The reason is developed in Section 6.2: a diff is what makes it possible to check a change rather than re-reading an entire codebase, and that matters more for the agent than it does for you.

If you do not use git yet, the following is a complete instruction to hand to an agent:

```
Initialise a git repository here, add a .gitignore that excludes the data
directory, logs, and LaTeX build files, and make an initial commit.
```

[2] https://github.com/claesbackman/AI-research-feedback, https://github.com/scunning1975/MixtapeTools, and https://claudeblattman.com/.

3.7 Managing the context window

An agent can hold only a limited amount of text in working memory at once — roughly a few hundred pages — and performance degrades as it fills. For a large empirical project with many scripts, logs, and output tables, this happens quickly. Three habits help.

Point at specific files instead of letting the agent read everything. Both systems support mentioning a file directly in the prompt (`@02_analysis.R fix the clustering in the main spec`), which is faster and cheaper than making it infer the project.

Start a new session for a new task instead of continuing one long conversation across unrelated work. This also has a verification benefit, which Section 6 returns to repeatedly: a fresh session is a fresh context, and fresh context is what independence is made of.

Check usage periodically. Both tools report how much of the window is consumed.

A well-written project-context file helps here too. Because it is read at the start of every session, you do not need to re-explain the project in each conversation, which keeps prompts short.

4 Choosing a model and controlling what it costs

Model choice is usually presented as a quality question. In day-to-day research work it is mostly a cost question, because the quality difference between tiers matters on a narrow set of tasks and the price difference applies to all of them. This section is about telling those tasks apart.

4.1 What you are actually paying for

There are two billing shapes. Under a subscription you pay a flat fee and the cost surfaces as a usage limit you hit in the middle of a task. Under metered API access you pay per token and the cost surfaces as a bill. Most academic users start on a subscription and should stay there until something forces the issue.

In either case the dominant quantity is *input* tokens. This surprises people, who reasonably assume they are paying for what the model writes. An agent resends the whole conversation on every turn, together with every file it has read along the way. A session that has opened twenty scripts pays for those twenty scripts again on each subsequent exchange, whether or not they are still relevant.

Two consequences follow, and they are the whole of the practical advice.

The first is that cost grows with session length even when the work does not. A long conversation is expensive at the end for reasons that have nothing to do with what you are asking at the end. The context discipline in Section 3.7 is therefore also cost discipline. The same habits produce both.

The second is that a stable prefix is cheap. Most vendors cache repeated context, so the parts of a session that do not change between turns cost less on re-read than they did the first time. That rewards a project-context file that stays put and an early conversation that you do not keep editing. It penalises the habit of re-pasting a slightly different version of the same instructions.

4.2 Matching the model to the task

Models come in tiers, and the price gap between the cheapest and the most capable is large enough to change how you work. The useful question is which tasks can tell the difference.

Cheap models are fine when the output is visibly right or visibly wrong. Converting a file format, reshaping a CSV, renaming variables across a project, writing a plotting script, drafting a commit message, running a script and reporting the row

counts. You can see the answer. If it is wrong, you will know within seconds, and the cost of the retry is trivial.

Capable models earn their price where the failure is invisible. Planning a panel construction, working out what a specification actually identifies, reviewing a diff for the silent shortcut, deciding whether a merge did what it was supposed to do. These are the tasks from Section 6, and they share the property that a wrong answer looks exactly like a right one.

The rule that falls out of this is short. Spend on judgement, economise on typing.

> **Do not economise on the review pass.** A cheap model reviewing a diff will find the things that were going to break anyway and miss the shortcut. That is the one place where saving money buys you nothing, because the error class you are hunting in Section 6 is precisely the one that requires understanding the analysis. If you are going to run a check, run it with a model capable of passing it.

4.3 Reasoning effort

Several tools expose a reasoning-effort or thinking-budget setting. Higher settings cost more and take longer, sometimes by a lot. The default is usually sensible for ordinary work.

A habit worth forming: raise the effort for the plan and lower it for the implementation. The plan is where the thinking happens and it is a page long, so the expensive setting is cheap in absolute terms. The implementation is mostly transcription of a decision already made. Raising effort for a long code-writing session spends the budget where it changes least.

Raise it again for a review pass over the main table, for the same reason.

4.4 Habits that cut the bill

Each of these appears elsewhere in this document for a correctness reason. They are collected here because they all happen to save money as well, which is worth knowing when you are deciding whether the discipline is worth it.

- **Point at files.** Letting an agent search a large project to find what it needs costs more than telling it. An `@`-mention of the file you care about replaces an exploration.
- **Start a new session for a new task.** A session carries its history. Two short conversations cost less than one long one covering the same ground.
- **Review diffs, not codebases.** The point in Section 6.2 was that a whole-project review produces worse findings. It also costs several times as much, because the whole project goes into the context to produce them.
- **Convert once and commit the result.** Re-converting the same 200-page PDF at the start of every session is a recurring charge for a job you finished months ago.
- **Ask for a plan before code.** A page of prose is cheap to produce and cheap to reject. Four hundred lines are expensive to produce, expensive to read, and expensive to replace when you decide the second step was wrong.
- **Do not let the agent run the expensive script.** Section 6.6 gives the correctness argument, which is the important one. The side benefit is that you are not paying for the model to watch several thousand lines of output go past.
- **Reserve the costly checks.** Reimplementation in a second language is the expensive rung of the ladder. Run it on the exhibits that go in the paper.

- **Batch the mechanical work.** Twelve robustness specifications in one unattended run cost less, in tokens and in attention, than twelve conversations that each re-establish what the project is.

4.5 Where the cost actually lands

For most academic users the token bill stays small next to the value of the time involved, and worrying about it in detail is a poor use of the time it saves. A subscription covers ordinary work. The reason to care about this at all is that the habits above do double duty.

Short sessions, small diffs, explicit file references, and a plan before an implementation are the same practices that keep an agent accurate. An expensive session and a session you cannot verify tend to be the same session, and they are expensive for the same reason: too much context, too little structure, and no point at which anyone decided what was supposed to happen.

5 Verification begins before the code is written

Section 6 sets out checks you can run on finished code. Every one of them assumes the code exists and asks how to establish that it is right. They are useful, and I run them. But the most reliable way to understand an analysis is to have been present while it was built, and that is cheaper than any of them.

Compare two sessions. In the first, you describe the whole pipeline in a paragraph, the agent works for six minutes, and hands you 400 lines across five files. Verifying that means reverse-engineering someone else's reasoning from its output, which is the position a referee is in, except that you are the author. In the second, you agree on a plan, the agent asks three questions you had not thought about, then writes one cleaning step, runs it, and shows you the counts. Both produce the same script. In the second you already know what it does, and the checks in the next section confirm what you already know.

The second session is also not slower in the end. Time saved by one long prompt is borrowed against the debugging session where you discover, three tables later, that the deflator was applied twice. The exception is mechanical work with a visible output, where you can see whether the result is right and none of this is needed.

A working shape for the loop, using panel construction as the example.

5.1 Ask for a plan, not code

A plan is a page of prose you can read in a minute, and disagreeing with it costs nothing. Disagreeing with a finished implementation costs a rewrite. Most tools have an explicit mode for this. If yours does not, say so in the prompt.

```
Before writing any code, describe how you would build the firm-year panel
from the raw files: which files, in which order, what the key is at each
stage, and where observations could be lost. Number the steps so I can
respond to them individually. Do not write code yet.
```

5.2 Make it ask you questions

An agent that has to ask whether firms changing industry mid-panel keep their original classification has surfaced a decision that would otherwise have been settled by whichever

default it picked, and that you would have discovered, at the earliest, when a co-author asked.

```
Before you start, ask me every question you need answered to do this
correctly. I would rather answer ten questions now than discover your
assumptions later. Include the ones where you could guess a reasonable
default - I want to know which defaults you would have chosen.
```

5.3 Work in steps small enough to check, and run each one

Code that has not been executed is a hypothesis. Ask for one step, have it run, look at the output, then continue. The counts after each step are the verification, and they arrive while the decision is still fresh enough to discuss.

```
Implement step 1 only. Run it, then report the number of observations and
unique firms, and show me the first ten rows. Stop there and wait for me
before starting step 2.
```

5.4 Ask why, not what

A summary of what the code does restates the code. The useful question is about the decisions — the places where something else was possible — because that is where your judgement is needed. Note that the second half of this prompt asks for something the agent has to run, not something it can assert.

```
List the three decisions in what you just wrote where a reasonable
person could have done something different, and what you chose in
each case. Then re-run the specification under each alternative and
give me the coefficients side by side. Do not tell me what you expect
the alternative to do - run it.
```

> **The questions are the deliverable.** Keep the ambiguities the agent raises and your answers to them in a `decisions.md` in the repository, with a one-line pointer to it from your project-context file, not the decisions themselves. This is the sample-definition documentation you would otherwise reconstruct from memory for the appendix eighteen months later.

6 Verifying what comes back

6.1 First principle: you have to read the code, or some version of it

The code is the analysis. You have to do some version of checking that it does what you actually want it to do. You can do that in many ways, and some are more efficient than others. If you use git you can read only the parts that change. You can have the agent present the changes along with an explanation. But you are responsible for the code once it produces results that appear in the paper, and so you have to do some of this work yourself.

I am still figuring out how to read code updates as I go along, and I do not know that I have the right approach yet. What is clear is the shape of the problem.

> **You are not looking for the errors that break things.** Agents are very good at iterating and figuring out what crashes, and they work autonomously to fix it. So you will get a table with output, and it will look plausible. It may still be wrong, because the agent took a shortcut. The errors you are looking for are the subtle ones that do not crash but lead to incorrect conclusions.

This is worth stating precisely, because it is the reason ordinary code review intuitions mislead here. The agent's own error-correction process runs to convergence on everything that produces a visible failure. What survives that process is, by construction, the set of errors that produce no visible failure. The output of a competent agent is therefore *selected* for the error class that is hardest for you to see. Fluent, plausible, and wrong is the dominant failure mode, not incoherence.

6.2 Use git to make verification tractable

Git matters here more for the agent than for you. If you ask an agent to review the code, it will look over all of the code and spread its attention across too many things. That invites generic answers, a lack of focus, and a larger bill. If it examines a diff instead, it knows exactly what is new, and can compare the change against code you have already checked.

So: commit before every session, so that there is a baseline to diff against. The rest of this section assumes you have one.

6.3 Check 1: a fresh agent attacking the diff

Launching a fresh agent or a fresh chat to evaluate the code is the easiest important check. The reason to use a fresh one is that the agent that wrote the code has too much context. Because it wrote the code, it has the whole conversation in mind, and it is not an independent arbiter of what is right. A fresh agent has no stake in the code and is more likely to find problems.

Two ways to do this: ask the current agent to launch a subagent to review the change, or start a new session yourself pointed at the diff.

One caveat on subagents: the parent writes their instructions and summarises the report back to you, so you read a filtered account. A new session that you brief yourself is the more isolated option.

> **Have the reviewer report, not fix.** A reviewer that edits as it goes hands you a second diff to verify, produced by an agent optimising for closing findings rather than for correctness. Read the findings, make it prove the ones that matter, and fix those yourself. Confident false positives are common enough that acting on an unexamined list will cost you more than it saves. You can also launch a further agent to check whether the problems it found are real.

6.4 Check 2: a model from a different developer

This is similar to Check 1 in that a fresh agent looks at the code, but sometimes there is value in a different model entirely. Two instances of the same model share training data and habits of thought, so they tend to make the same mistakes.

Running the diff past a model from a different developer breaks that correlation. The logic is the same as sending a paper to two referees from different subfields rather than to two students of the same advisor. The errors have to be independent for the second look to be worth anything, and vendor is the cheapest axis of independence available.

Give both tools the same project context. The two systems read different instruction files. Keep the substance in one file and have the other point to it, so that the second opinion works from the same sample definitions as the first.

6.5 Check 3: make the agent quiz you

The checks so far verify code using other models. But you are also supposed to be in the loop, and that is the easy part to skip. A quiz is a good way to establish that you actually understand what happened.

The idea is adapted from Geoffrey Litt's *explain-diff* skill.[3] The agent writes a self-contained HTML page explaining a change — background, the core intuition with toy examples and diagrams, a walkthrough of the code — ending in five multiple-choice questions with immediate feedback. The instruction at the heart of it:

```
Quiz: Come up with five questions that test the reader's knowledge
of this PR. This should be medium difficulty, difficult enough that
you actually need to understand the substance of the PR to answer
them, but not gotchas. The goal is to help the reader make sure that
they've actually understood. These should be presented as interactive
multiple-choice questions, and when the user clicks, it tells them
whether they were correct and gives feedback.
```

Appendix A contains a version adapted for an empirical paper rather than a software change, with the questions weighted towards empirical consequences — which observations enter the sample, what the coefficient now identifies, what would change if an assumption failed — rather than syntax.

Run it in a new session, not the one that made the change. Starting fresh forces the agent to read the files instead of recalling the conversation, which is why the skill opens by telling it to.

The thirty-second version. When a full explainer is overkill, open a fresh session and ask: *Read the last commit and the files it touches, then ask me three questions about what it changed. Do not tell me the answers until I have answered. If my answer is wrong, say so directly — do not tell me I was close.*

6.6 Check 4: reimplement the analysis in another language

This is a check Scott Cunningham has advocated for. It is the most expensive one here and also the strongest: you ask for the analysis in a different programming language and see whether you get the same output.

It is strong because it involves the whole chain. If errors across languages are independent, differences in output reveal them. An independent implementation catches the case where the do-file faithfully implements something other than what the paper says it does. That is the failure mode no amount of reading the do-file will surface, because the do-file is internally consistent. Use a different model as well, for the reason in Check 2. And note that this check is getting cheaper over time, as agents get better at writing code.

[3] https://www.geoffreylitt.com/

6.6.1 Isolating the second implementation

This check can happen organically. Agents are prone to reaching for Python to generate intermediate results, for instance. If you separately ask for a Stata pipeline and the numbers match, you get the check for free. Verification is not a one-time event so much as a continuous process.

But it is worth thinking about how to run it deliberately, and doing that requires taking seriously how these models work. If you ask the agent to *"ignore what you know about `02_analysis.do`"*, you are asking it to un-condition on something already in its context, which is not an operation it can perform. The context is always loaded. Independence has to be enforced by removing the contaminating context, not by requesting that it be disregarded. **This is the single most important practical point in this document**, and it generalises well beyond this check: every claim of independence in Sections 6.3–6.6 is only as good as the physical isolation behind it.

Concretely: create a new folder and start a fresh session in it. Copy the specification description out of the paper into `spec.md` and delete the results from it. Write, or have an agent write, a `README.md` describing the data and variables. Then:

```
Read verify/spec.md, which describes a specification, and data/README.md,
which describes the raw data. From those two documents only, write an R
script that estimates the specification starting from the raw files in
data/raw/. Save it as verify/analysis_check.R.

Where the description is ambiguous, do not guess. Stop and list the
ambiguities and I will resolve them.

Do not run it. I will run it myself and tell you what it returns.
```

Running it yourself matters, and not only for the token cost. **An agent that runs its own script and sees an implausible number will debug towards plausibility, and plausibility is exactly what you are trying to test.** The instruction to stop at ambiguities is also useful in its own right, because the ambiguities it surfaces are often ones you want to resolve in the paper too.

> **Reserve this for what matters.** Since it is costly, run it on the main results — the table and figure exhibits in the paper — and on any step where a mistake would be invisible in the output: a complicated panel construction, a hand-rolled event-time definition, an index built from several sources.

7 Checks that cost almost nothing

The four checks above are deliberate acts. These are habits.

7.1 Verify the data, not only the code

Correct-looking code operating on the wrong sample produces wrong results, and no amount of code review will catch it. Make sure you know what the data looks like. A sample-construction table is a simple way to do this. Check that merges behave as expected, and look at the distribution of constructed variables.

7.2 Compare against an external benchmark

We all do this when working out whether results "make sense", and we should keep doing it for AI-generated output. This is almost certainly already part of your workflow,

so the marginal cost is near zero. It is external validation, which catches a class of error that no internal check will.

7.3 Check the numbers in the prose against the numbers in the tables

Coefficients get quoted in the abstract and the introduction, the specification then changes, and the writing does not get updated. This is a tedious check for a human and a trivial one for an agent. It is included in my *review-paper* skill.

7.4 Verify your references

Models still invent citations. There are automated checkers for this now — Reviewer3, among others — and it takes a minute.

8 Working with restricted data

Most of this document is agnostic about where the work happens. This section is not, and for anyone using administrative registers it is the part that determines whether the rest is usable at all.

8.1 What actually leaves your machine

Start with the misconception that causes the most trouble. Running an agent *locally* does not keep your data local. The tool runs on your machine. The model does not. Every file the agent opens, every excerpt it quotes back, and every word you type is transmitted to the vendor for inference.

The word *local* describes where the editor window is. It says nothing about where the file contents go. A researcher who has carefully avoided the web surface, installed the command-line tool on a secure laptop, and pointed it at a folder of register extracts has transmitted the register extracts.

There are two real exceptions. Models running entirely on your own hardware keep everything in place, and some institutions run approved deployments inside their own boundary. Both exist. Both are meaningfully worse at this work than the hosted frontier models. That tradeoff is real, and worth weighing properly rather than waving away.

So the first task, before the first session, is to work out which of three situations you are in.

8.2 Three regimes

Regime A: the data may be transmitted. Public data, purchased data, or licensed data whose agreement permits third-party processing. The ordinary workflow in this document applies without modification. It is still worth checking the vendor's retention and training terms, and using a zero-retention or institutional arrangement if your university has negotiated one.

Regime B: the data may not be transmitted, but code may. This is the common case for administrative data held under a data-use agreement on a machine you control. You can use an agent productively. You cannot point it at the data. Section 8.3 is about how to work in this regime, and it is the one most readers of this document will be in.

Regime C: nothing leaves. A secure environment with no outbound network, which is how most national statistics agencies and many secure enclaves are built. No hosted agent is reachable from inside. Unless your institution provides an approved deployment within the boundary, the agent stays outside and only your own hands go

in. Everything below about synthetic data still applies, and the transfer step becomes whatever your environment's approved file-transfer process is.

8.3 The synthetic-data pattern

This is the workhorse for Regime B, and it works well enough that the loss relative to unrestricted work is smaller than people expect.

Build a mock dataset with the same shape as the real one. Same file layout, same variable names, same types, same value ranges, same missingness structure, same key structure including whatever duplication the real keys have. Populate it with random values. Then give the agent the mock data and develop the entire pipeline against it. The agent can read files, run code, see the errors, and iterate, which is the whole benefit of an agent and none of it requires real observations. When the script is finished, move it into the restricted environment and run it there yourself.

Writing the generator is itself a good first task to hand the agent, working from the data dictionary described below.

> **Expect the first real run to fail, and treat that as information.** Mock data has none of the pathologies that make real data hard. No duplicate identifiers you did not know about, no sentinel values coded as 9999, no year where the variable definition quietly changed, no municipality that merged. Those are exactly the places a pipeline breaks. The run inside the environment is a real test, so budget for it failing.

8.4 The data dictionary is the interface

What you send instead of data is a description of it. A markdown file listing the files, the variables, their types and units, the coverage, the key structure, and the quirks you already know about. This is what the agent reasons from, and the quality of the pipeline it writes depends almost entirely on the quality of this document.

Check that the dictionary is itself releasable. A variable list is usually fine. Value ranges can disclose more than they appear to, and a frequency table with small cells may breach the same disclosure rules that govern your output. This is a judgement your data-use agreement governs, and it is worth asking the data provider instead of guessing.

The side benefit is that this is documentation you needed anyway, and writing it early is easier than reconstructing it for the appendix later.

8.5 Output is disclosure too

The easiest rule to follow is the one about not uploading the microdata. The easier one to break is everything downstream of it.

Pasting a regression table into a chat to ask what it means transmits results. Aggregates are often permitted under a data-use agreement and small cell counts often are not, so the question to ask is whether the output clears your disclosure rules. A traceback from a crashed script can carry a row of raw data inside the error message, and debugging a crash is the single most natural thing to hand an agent. A scatter plot of individual observations is microdata drawn as a picture.

None of this is exotic. It just happens at moments when you are thinking about something else.

8.6 Where the transcript lives

Agents write session logs, caches, and history files to disk. If a session touched restricted material, those files are now copies of restricted material in a location your data-use agreement almost certainly does not list.

Find out where your tool stores conversation history and either keep it inside the approved boundary or turn it off. The same applies to the project-context file and to the `decisions.md` from Section 5, both of which accumulate specifics about the data and both of which get committed to git. Keep sample definitions in them and keep actual values out.

Set the `.gitignore` to exclude the data directory before the first commit. Git remembers, and removing a file later does not remove it from the history.

8.7 The verification ladder under restriction

The checks in Section 6 survive, with adjustments.

Check 1, a fresh agent on the diff, works unchanged in Regime B. A diff of code is code.

Check 2, a second vendor, works, at the cost of a second set of terms to review and a second account to get approved by whoever approves such things at your institution. Start that conversation early, because it is slower than the technical work.

Check 3, the quiz, works unchanged. It operates on the code and the diff.

Check 4, reimplementation, is the awkward one. It needs a specification and a description of the data, and both are derived from restricted material. In Regime B, write `spec.md` from the paper, not from the data, and reuse the dictionary you already built. In Regime C the honest version is a second implementation written outside the environment, carried in, run there, and compared only on the coefficients you are permitted to bring out.

The cheap habits in Section 7 are unaffected, and they become relatively more valuable, because every one of them runs entirely inside the boundary.

8.8 Before the first session

A short checklist, done once, deliberately, and not in the middle of a task.

1. Read the data-use agreement for what it says about third-party processing. If it is silent, ask the provider rather than assuming silence is permission.
2. Decide which regime you are in and write it down where coauthors can see it.
3. In Regime B or C, build the mock data and the data dictionary before anything else.
4. Set the `.gitignore` before the first commit.
5. Find out where the tool writes its logs and history.
6. Agree all of the above with your coauthors. One coauthor pasting the sample into a chat window undoes every other precaution on this list, and they will not know they did anything unusual.

9 Where this is going

Three things seem worth saying about the direction of travel, offered as expectations and not predictions.

The checks in Section 6 get cheaper over time and the work they check gets cheaper faster. Reimplementation in a second language was an unreasonable ask three years ago

and is now an afternoon. That is good, but it means the ratio of produced-to-verified analysis keeps rising unless verification becomes routine work and not a heroic effort. The habits in Section 7 and the loop in Section 5 matter more than the expensive checks precisely because they scale.

The profession has not settled what it is reasonable to expect. A referee cannot currently ask what was AI-generated and what was not, and would not know what to do with the answer. My own guess is that the norm that eventually forms is not about documentation of checking, closer to what we already expect for a replication package than to a conflict-of-interest statement. The `decisions.md` file in Section 5 is a small bet on that.

And the material in Appendix B will be wrong before the material in Sections 5 through 7 is. That is the reason for the split.

A Skills referred to in the text

A.1 *quiz-me* — explain a change and test that I understood it

Adapted from Geoffrey Litt's *explain-diff* skill for an empirical paper rather than a software change. Save as `~/.claude/skills/quiz-me/SKILL.md`, which makes it `/quiz-me`. Run it in a new session, not the one that made the change.

```
---
name: quiz-me
description: Explain a change to the analysis and quiz me on whether
  I understood it. Use when I have asked for a change to cleaning or
  estimation code and need to verify my own understanding.
disable-model-invocation: true
---

Work only from the code and the diff. If this conversation contains
an earlier account of the change, ignore it and read the files.

Produce a single self-contained HTML file explaining the specified
change, ending in a quiz. Save it outside the repository with a
filename starting with today's date in YYYY-MM-DD- format, then
tell me the path.

Sections:

1. What this code did before. Explore the surrounding scripts, do
   not rely on the diff alone.
2. What changed and why, in plain language. No code in this section.
3. Consequences for the results. Which sample, which coefficients,
   which tables are affected, and in which direction. Say so
   explicitly if the answer is none.
4. Walkthrough of the changed code, grouped by purpose rather than
   by file.
5. Quiz. Five multiple-choice questions, medium difficulty - hard
   enough that answering requires understanding the substance, not
   gotchas. At least two must be about empirical consequences
   rather than syntax: which observations enter the sample, what
   the coefficient now identifies, what would change if an
   assumption failed. Distractors must be plausible
   misunderstandings, and must match the correct answer in length,
   grammar, specificity, and confidence. Randomise the position of
   the correct answer independently for each question. On click,
   report whether the answer was correct and explain why, including
   why each wrong option is wrong. For each question, cite the file
   and line the answer rests on.

Style: clear prose, concrete toy examples with made-up numbers, and
simple HTML diagrams rather than ASCII art. Code in <pre> tags.
Embed all CSS and JavaScript so the file works offline.
```

A.2 Note on automatic invocation

An agent may run a skill on its own when the description matches what you asked for. For a verification skill this is usually unwanted, because the point is that you chose to run it. The `disable-model-invocation: true` line above prevents it.

B Tool-specific reference

This appendix decays. It was written from official documentation checked in May 2026. Product names, file conventions, and mode vocabulary in this area change every few months. Check the current version at claesbackman.github.io/ai-guides.html before relying on any specific detail below. Nothing in Sections 5–7 depends on it.

B.1 Claude Code and Codex, side by side

Both tools read a project, edit files, run commands, review diffs, and help with R, Stata, Python, LaTeX, and data-cleaning glue code. The real differences are in workflow: where instructions live, how reusable workflows are invoked, how permissions are managed, and how local work connects to cloud work. This is a workflow comparison, not a model benchmark.

	Codex (OpenAI)	**Claude Code (Anthropic)**
Project instructions	`AGENTS.md`. Reads global and project files before working, layering root and nested instructions, with closer files taking precedence.	`CLAUDE.md`. Persistent memory files at session start, plus auto memory and path-specific rules.
Reusable workflows	Skills invoked by mention, e.g. `$robustness`. Usually in `.agents/skills/` or `~/.agents/skills/`.	Skills invoked as slash commands, e.g. `/robustness`, in `.claude/skills/`. Legacy `.claude/commands/` still works.
Slash commands	Mainly session controls: `/cloud`, `/local`, `/review`, `/status`.	Broad surface: `/init`, `/memory`, `/permissions`, `/compact`, `/context`, and more.
Approvals	Three IDE modes: *Chat*, *Agent*, *Agent (Full Access)*. Default agent mode edits and runs commands in the working directory, asking before network or outside-directory work.	Named permission modes — `default`, `acceptEdits`, `plan`, `bypassPermissions` — with fine-grained allow/ask/deny rules.
Cloud work	Foregrounded. Send larger jobs to a cloud environment from the IDE, monitor, and apply diffs locally. The clearest differentiator.	Primarily local in VS Code. Web, desktop, and terminal surfaces exist, but the extension emphasises interactive work and resumable conversations.
Subagents	Explicit subagent workflows and custom agents, surfaced mainly in the app and CLI.	Specialised subagents with their own context, prompts, tools, and permissions. Delegates when a task matches.
Hooks	Deterministic scripts on lifecycle events, behind a feature flag in `config.toml`.	Mature automation surface with many lifecycle events, shell commands, HTTP endpoints, and prompt/agent hooks.

	Codex (OpenAI)	Claude Code (Anthropic)
Reasoning	Explicit reasoning-effort control in the IDE. Start at medium and raise it for complex debugging or ambiguous empirical logic.	Model selection in the panel, with plan review and checkpoints in the interface.

B.1.1 Which to use

Codex fits better when your workflow already lives in OpenAI accounts, when you want a strong local-to-cloud path for larger tasks, when you prefer `AGENTS.md` and OpenAI-style skills, or when you expect to work across IDE, CLI, web, and GitHub surfaces.

Claude Code fits better when you already use Anthropic accounts, when you want a mature VS Code interface with plan review and checkpoints, when your team already has `CLAUDE.md` or `.claude/skills/`, or when you want detailed permission modes as part of everyday work.

Practical recommendation: keep both installed for a while if your budget allows. Use one as a daily driver and the other as the second opinion in Check 2. The second subscription buys you the independence that check depends on.

B.1.2 Migrating between them

The conceptual pieces transfer. The file names and invocation change. Move durable project context from `CLAUDE.md` into `AGENTS.md`, keeping facts rather than prose. Convert skills by moving

```
.claude/skills/robustness/SKILL.md
    ->  .agents/skills/robustness/SKILL.md

/robustness
    ->  $robustness
```

and changing the invocation accordingly. Translate permission modes into the simpler three-way IDE switch. Do not try to map every slash command one-to-one: Codex slash commands are mostly session controls, so reusable research procedures belong in skills instead.

B.2 VS Code extensions for empirical research

Extension ID	What it does
`tmonk.stata-workbench`	Runs Stata from the editor and exposes an interface the agent can drive to execute commands, inspect variables, and read stored results. Built by Thomas Monk, LSE.
`ms-python.python`	Core Python support. Required for anything Python-related.
`ms-python.vscode-pylance`	Type-aware language server. Gives the agent better static context: variable types, signatures.

Extension ID	What it does
`ms-python.debugpy`	Debugger. Breakpoints and mid-run inspection, for tracking down why a merge or regression loop misbehaves.
`ms-python.vscode-python-envs`	Virtual environment management, so different papers can use different package versions.
`james-yu.latex-workshop`	Compile on save, side-by-side PDF preview, inline errors, citation autocomplete.
`yzane.markdown-pdf`	Markdown to PDF in one command, for memos and referee responses.
`mechatroner.rainbow-csv`	Colour-codes CSV columns and adds a lightweight query tool for filtering rows without loading the file.
`tomoki1207.pdf`	Renders PDFs inline, for reading codebooks alongside code.

B.2.1 LaTeX build-file cleanup

By default the LaTeX extension leaves auxiliary files next to your source. To have them removed after each successful build, add to `.vscode/settings.json`:

```
"latex-workshop.latex.autoClean.run": "onBuilt",
"latex-workshop.latex.clean.method": "glob",
"latex-workshop.latex.clean.fileTypes": [
  "*.aux", "*.bbl", "*.blg", "*.log",
  "*.out", "*.toc", "*.fls", "*.fdb_latexmk",
  "*.nav", "*.snm", "*.vrb"
]
```

Set `autoClean.run` to `"never"` if you would rather keep them, which is useful when debugging bibliography or cross-reference errors.

C Editing LaTeX locally while co-authors stay on Overleaf

Overleaf is good for collaboration and limited as an editor: no agent integration, no proper diffing, no terminal. The fix is to write locally and let Overleaf stay in sync. Two routes work, and both require Overleaf Premium, which many universities provide through an institutional login.

Use GitHub if you ever want to run `git log` on your paper, which also means the diff-based checks in Section 6 apply to your prose as well as your code. **Use Dropbox** if a co-author is editing in Overleaf at the same time you are typing locally.

	GitHub	**Dropbox**
Sync	Manual push and pull	Automatic, a few seconds
History	Full git history, branches, PRs	Dropbox revisions only
Best for	Solo work, versioned drafts, referee revisions	Live co-authoring with someone in Overleaf
Conflicts	Git merge	"Conflicted copy" files

Pick one per project. Running both on the same Overleaf project creates duplicate-file chaos. If you switch, unlink the old one first.

C.1 GitHub route

In Overleaf, open the project menu and choose Integrations → GitHub → Export Project to GitHub, then create the repository. Clone it locally. GitHub Desktop gives you commit, push, pull, and branch without the command line, which is the default I would recommend for writing, though the CLI works identically if you prefer it. Open the folder in VS Code, install the LaTeX extension for local preview, and work normally. Commit and push when you want Overleaf to see the change. In Overleaf, pull GitHub changes to bring them in. The reverse direction, for co-author edits made in Overleaf, is a push from the same menu.

Accept the TeX `.gitignore` template when setting up the repository. It keeps compiled output out of the history.

C.2 Dropbox route

Install Dropbox and let the initial sync finish. In Overleaf, link Dropbox Sync. Overleaf creates an `Apps/Overleaf/` folder containing one subfolder per project. Open the project folder locally and edit. Each save propagates within seconds, in both directions. Compile in Overleaf, or locally with `latexmk -pdf`.

C.3 Things that will catch you out

- **The first sync overwrites.** Make sure both sides agree on initial contents before linking. If one side is empty, the other wins, and not always in the direction you expect.
- **GitHub sync is not instant.** Overleaf only pulls when someone clicks. Tell co-authors to pull after you push.
- **Dropbox conflicts are silent.** Simultaneous edits produce a `(conflicted copy).tex` file and no warning. Skim the folder occasionally.
- **Dropbox notifications are noisy.** The auto-sync produces frequent update messages. This is why I switched to GitHub.

- **Local compilation may need extra packages.** Overleaf ships a large TeX Live install. If the paper compiles there but not locally, install the missing packages with `tlmgr`.
- **Figure paths must stay inside the project.** Overleaf does not allow parent-directory references like `\includegraphics{../Figures/fig1.pdf}`. Keep figures in a subfolder of the project root.

D Sources and further reading

The verification checks in Section 6 are collected from my own experience and from others. Geoffrey Litt on understanding as the new bottleneck, and the *explain-diff* skill that Section 6.5 and Appendix A adapt. Paul Goldsmith-Pinkham on integration and collaboration in AI research work, which is where the emphasis on diffs in Section 6.2 comes from, and on writing with AI assistance. Scott Cunningham, who is the person I associate with the cross-language reimplementation check in Section 6.6.

For the broader picture of what these tools can do in economics, as opposed to how to check them, Anton Korinek's survey in the *Journal of Economic Literature* and its periodic online updates are the standard reference.

The resources page at claesbackman.github.io/resources.html collects writing from other economists on AI-assisted research — including Benjamin Golub's overview of modern AI tools for economics research, Pedro Sant'Anna and Aniket Panjwani on Claude Code with Stata, and Kevin Bryan's guide to AI, git, and LaTeX — alongside the broader hidden curriculum on writing, presenting, and coding.

Version 1.0 • August 2026. Assembled from the guides at claesbackman.github.io/ai-guides.html, where the tool-specific material is kept current. This document may be freely shared. Corrections and additional checks are welcome at claes.backman@gmail.com.